

IoT PROJECT

Smart Classroom Focus Monitor

ระบบ IoT สำหรับตรวจสอบความพร้อมของห้องเรียน ด้วย Edge AI + Cloud Dashboard

DHT22

LDR

PIR

Node-RED

InfluxDB

Blynk

ThingSpeak

ปัญหาและแนวคิด

เหตุผลที่นำ IoT มาใช้ในห้องเรียน

ปัญหาที่พบ

สภาพห้องไม่เหมาะสม

ห้องเรียนอาจร้อน มีด หรือไม่มีคนใช้งานแต่ยังเปิดระบบไว้

ผู้ดูแลไม่เห็นสภาพ real-time

ผู้สอน/ผู้ดูแลไม่สามารถติดตามสภาพห้องเมื่ออยู่นอกห้องได้

ขาดข้อมูลย้อนหลังเพื่อปรับปรุง

ไม่มีข้อมูลช่วยวิเคราะห์ช่วงเวลา que สภาพห้องไม่เหมาะสม

แนวคิดหลัก

ตรวจสอบสภาพห้อง → วิเคราะห์ความพร้อม

→ แจ้งเตือน → ควบคุมอัตโนมัติ

ระบบทำงานวนซ้ำเพื่อปรับสภาพห้องให้เหมาะสมเสมอ

ขั้นตอนการทำงาน

1

Monitor — ตรวจสอบสภาพห้อง

เซ็นเซอร์เก็บข้อมูล real-time: อุณหภูมิ แสง ความชื้น การเคลื่อนไหว

2

Analyze — วิเคราะห์ความพร้อม

ประมวลผลข้อมูลเทียบกับมาตรฐาน หาความผิดปกติของสภาพห้อง

3

Alert — แจ้งเตือนผู้สอน/ผู้ดูแล

ส่งแจ้งเตือนผ่านแอปหรือ dashboard เมื่อพบสภาพไม่เหมาะสม

4

Control — ควบคุมอุปกรณ์อัตโนมัติ

สั่งเปิด/ปิดแอร์ ไฟ พัดลม ตามเงื่อนไข เพื่อประหยัดพลังงาน

วัตถุประสงค์ของระบบ

เป้าหมายที่สอดคล้องกับเกณฑ์การประเมิน

เป้าหมายหลัก 5 ประการ

- 1 อ่านค่าอุณหภูมิ ความชื้น แสง และการมีคนในห้อง
- 2 คำนวณ Focus Score ประเมินความเหมาะสมต่อการเรียน
- 3 เก็บข้อมูลลง Time Series DB และแสดงกราฟย้อนหลัง
- 4 ควบคุม RGB LED, Buzzer และ Servo ทั้ง local/remote
- 5 แจ้งเตือนผ่าน Telegram เมื่อสภาพห้องผิดปกติ

ครอบคลุม 4 ด้านสำคัญ

Local Dashboard

แสดงค่าและควบคุมจาก
Node-RED Dashboard

Remote Cloud

ดูข้อมูลจากภายนอกผ่าน
Blynk + ThingSpeak

Edge AI

คำนวณ Focus Score + ตรวจ
anomaly ที่ Edge

Notification

ส่งแจ้งเตือนผ่าน Telegram
bot

อุปกรณ์ที่ใช้

Sensors ≥ 2 และ Actuators ≥ 2 ตามโจทย์

SENSORS — รับข้อมูลจากสภาพแวดล้อม

DHT22

Temperature / Humidity

วัดอุณหภูมิและความชื้นในห้อง

LDR / GY-302

Light

วัดความสว่างของแสงในห้อง

PIR

Motion

ตรวจจับการเคลื่อนไหวของคน

ACTUATORS — ตอบสนองตามคำสั่ง

RGB LED

Status Indicator

แสดงสถานะห้อง: เขียว/เหลือง/แดง

Buzzer

Local Alert

ส่งเสียงเตือนเมื่อพบความผิดปกติ

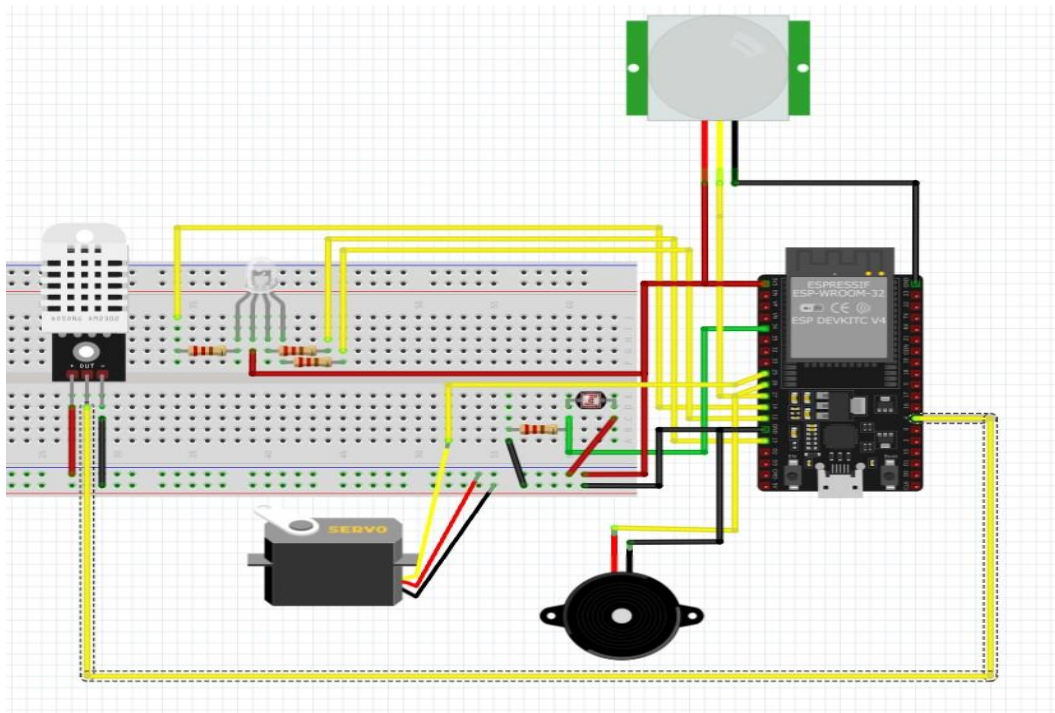
Servo

Curtain Demo

จำลองการเปิด/ปิดม่านอัตโนมัติ

Wiring Summary

การต่อวงจรของ ESP32 กับเซ็นเซอร์และ Actuators



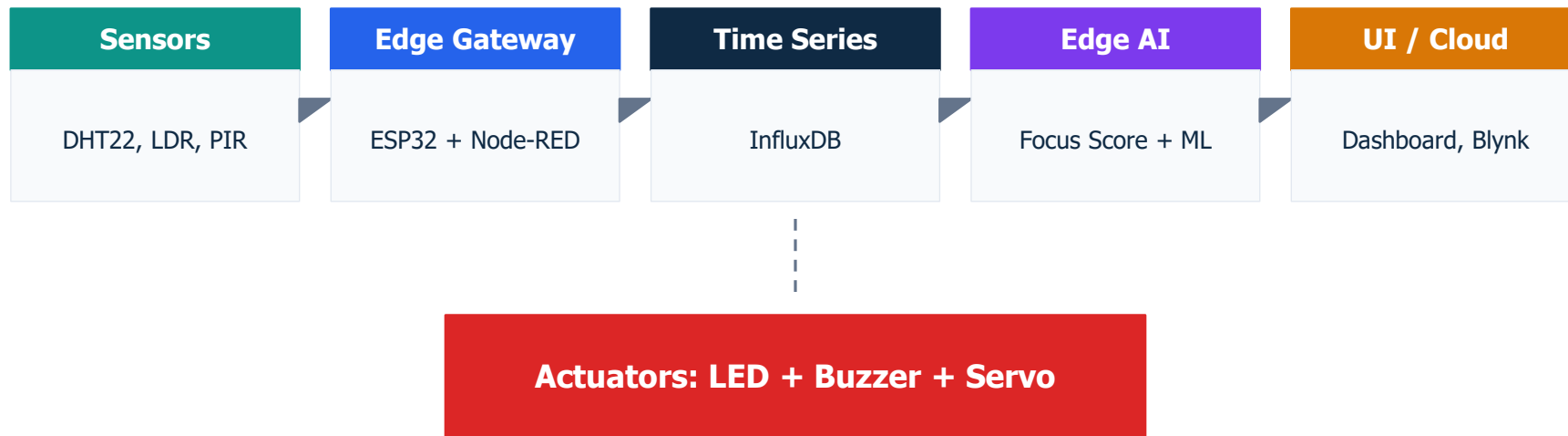
การเชื่อมต่อหลัก

● DHT22	GPIO 4
● LDR	GPIO 34 (ADC)
● PIR	GPIO 27
● RGB LED	GPIO 25, 26, 33
● Buzzer	GPIO 13
● Servo	GPIO 14 (PWM)

ใช้ breadboard เป็นจุดรวมการต่อสาย และต่อตัวต้านทานให้ RGB LED ทุกขา เพื่อป้องกันความเสียหาย

System Architecture

Sensor → Edge → Database → AI → UI/Cloud → Actuator

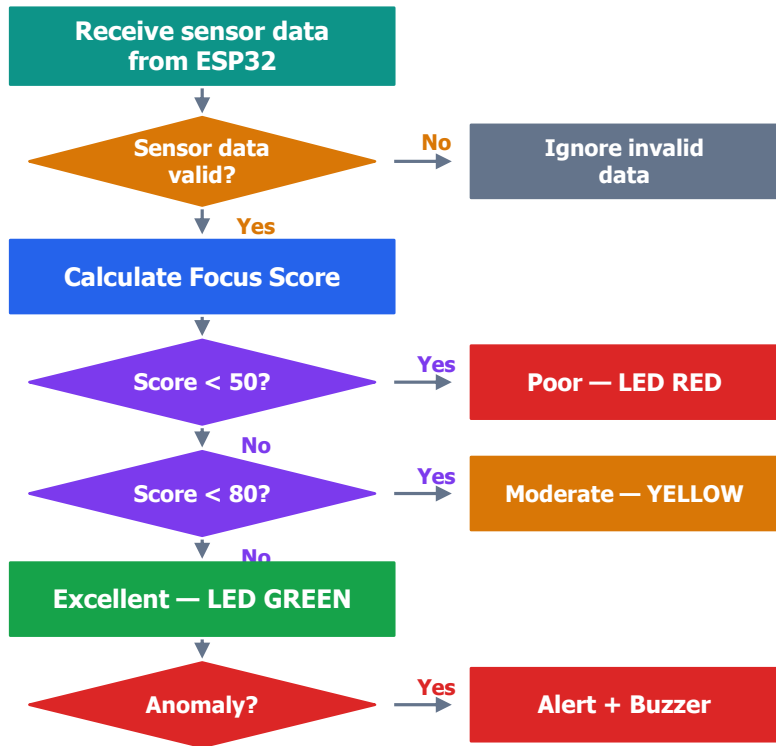


การไหลของข้อมูล

ESP32 อ่านค่า sensor → ส่งผ่าน MQTT ไปยัง Node-RED → ประมวลผล Edge AI คำนวณ Focus Score → บันทึกลง InfluxDB และส่งต่อ Dashboard/Cloud → ส่งคำสั่งกลับไปควบคุม Actuators

Flowchart การทำงาน

ลำดับขั้นหลักของระบบตั้งแต่รับข้อมูลถึงส่งออก

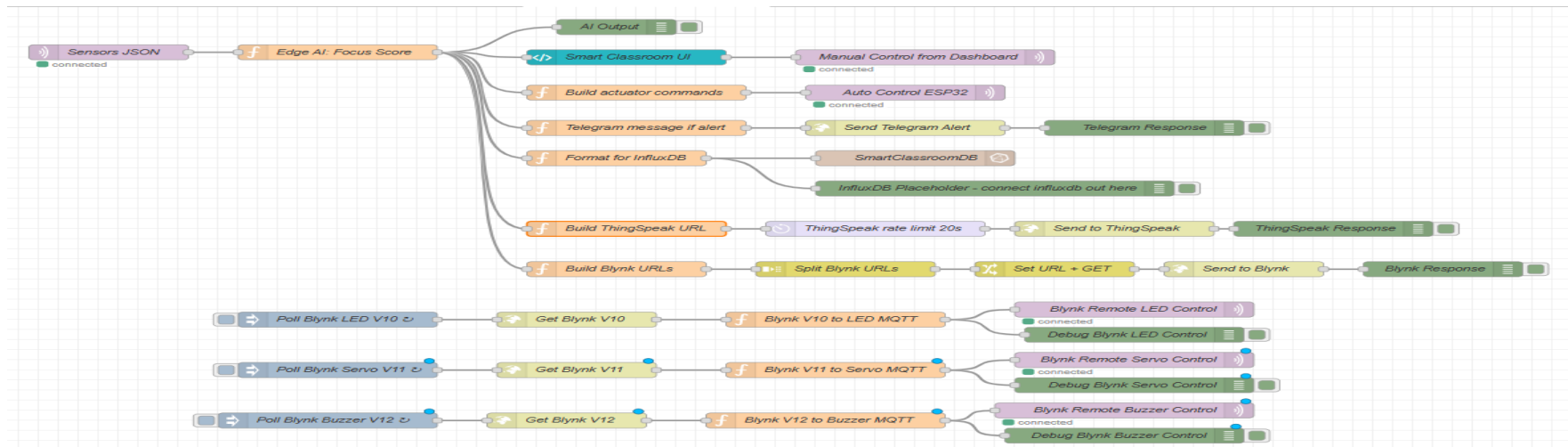


ขั้นตอนหลักของระบบ

- 1 รับข้อมูลจาก ESP32**
Node-RED รับ payload จาก MQTT
- 2 ตรวจสอบ valid**
ถ้าค่าผิด → ข้าม payload ถัดไป
- 3 Calculate Focus Score**
แปลง temp + light + motion เป็น 0-100
- 4 Classify Status**
Poor / Moderate / Excellent + เปลี่ยนสี LED
- 5 Anomaly Detection**
Temp > 30°C หรือมีคน แต่มีคน → alert
- 6 Edge AI ทำนายแนวโน้ม**
ใช้ 5 ค่าหลัง predict trend
- 7 ส่งออก**
Dashboard / InfluxDB / Cloud / Telegram

Node-RED Flow

Edge gateway สำหรับรับข้อมูล ประมวลผลและส่งต่อ



Input

- รับ Sensors JSON ผ่าน MQTT
- ส่งเข้า Edge AI function เพื่อคำนวณ Focus Score และตรวจ anomaly

Process

- Build actuator commands → ส่งกลับให้ ESP32 ทำงาน auto/manual
- Format ข้อมูลเพื่อเก็บลง InfluxDB

Output

- ส่งข้อมูลไป ThingSpeak + Blynk (cloud dashboard)
- Telegram alert เมื่อพบความผิดปกติ

Time Series Database

InfluxDB สำหรับเก็บข้อมูลย้อนหลังเพื่อวิเคราะห์

โครงสร้างข้อมูล

Database

classroom_iot

Measurement

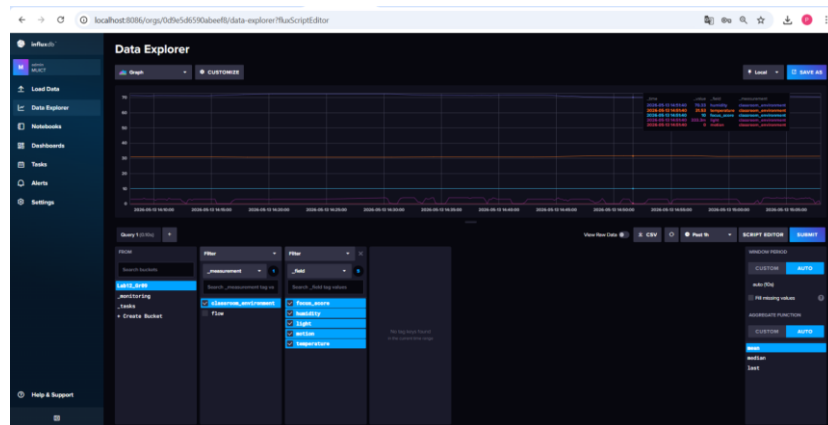
classroom_environment

Fields

temperature, humidity, light, motion,
focus_score

Window

10 วินาทีต่อจุด (auto)



ทำไมต้อง Time Series DB?

บันทึกข้อมูลตามเวลา

เก็บค่าทุกตัวพร้อม timestamp อัตโนมัติ

Query รวดเร็ว

ดึงข้อมูลช่วงเวลาที่ต้องการได้ทันที

วิเคราะห์ย้อนหลัง

หาช่วงเวลาที่ยังไม่เหมาะสมเพื่อปรับปรุง

Installation & Database Setup

การติดตั้งและตั้งค่า Node-RED + InfluxDB

ขั้นตอนการติดตั้ง

1

ติดตั้ง Node.js + Node-RED

```
npm install -g node-red
```

2

เพิ่ม Dashboard module

```
npm install node-red-dashboard
```

3

เพิ่ม InfluxDB nodes

```
npm install node-red-contrib-influxdb
```

4

ติดตั้ง InfluxDB v2

Docker หรือ Local installer

5

สร้าง Bucket ใน InfluxDB

Org: MUICT / Bucket: Lab12_Gr09

Node-RED InfluxDB Configuration

influxdb out node

Server: SmartClassroomDB

Organization: MUICT

Bucket: Lab12_Gr09

Time Precision: Milliseconds

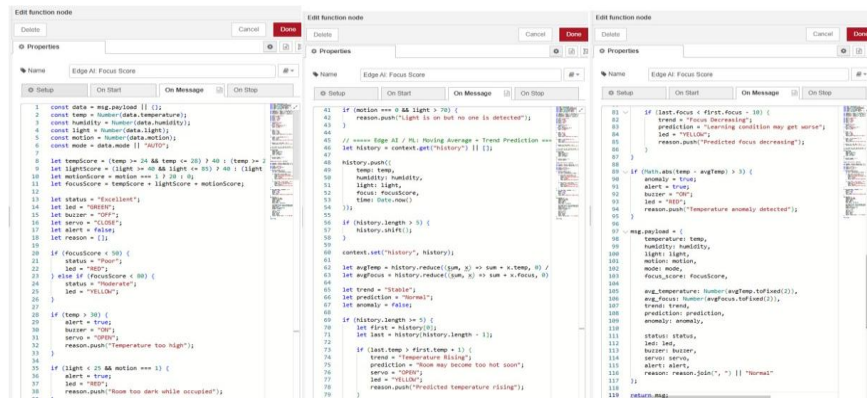
Format function

กำหนด measurement และ payload ก่อนส่งเข้า InfluxDB

Node-RED Function Nodes Setup

การตั้งค่า 3 function nodes สำคัญของระบบ

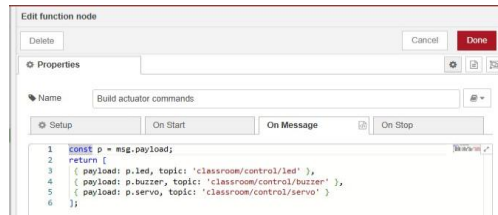
1. Edge AI: Focus Score



หน้าที่หลัก

- คำนวณ **Focus Score** จาก temp + light + motion
- จัด **Status**: Poor / Moderate / Excellent + สี LED
- เก็บ **history 5** ค่า ใน context สำหรับ ML
- ตรวจสอบ **trend + anomaly** และ predict ล่วงหน้า

2. Build actuator commands



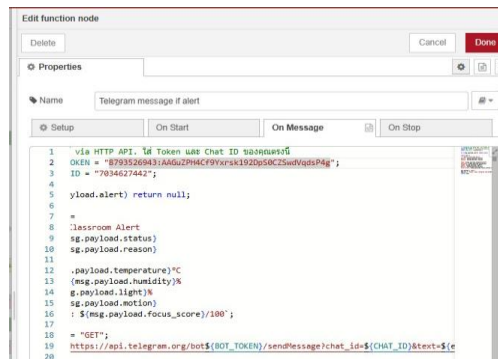
หน้าที่

แตก payload เป็น
3 MQTT topics:

- control/led
- control/buzzer
- control/servo

ESP32 subscribe และทำงาน

3. Telegram message if alert



หน้าที่

ส่งข้อความเข้า
Telegram bot

เมื่อ **alert = true**
ต้องใส่:

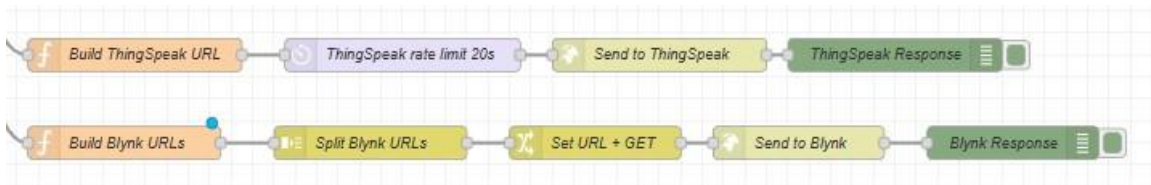
- BOT_TOKEN
- CHAT ID

ผ่าน HTTP GET /sendMessage

Cloud Connection Setup

วิธีเชื่อมต่อ Blynk + ThingSpeak จาก Node-RED

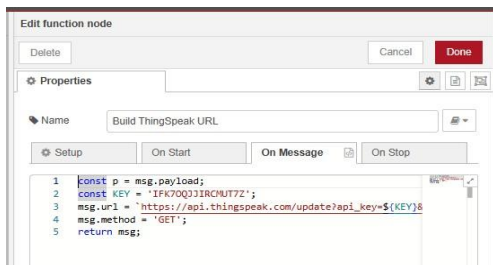
Node-RED Flow ส่งข้อมูลขึ้น Cloud



หลักการทํางาน

function node สร้าง URL แล้วส่งผ่าน HTTP request ไป cloud — ThingSpeak จำกัด 20 วินาที/ครั้ง, Blynk ส่งทีละ field

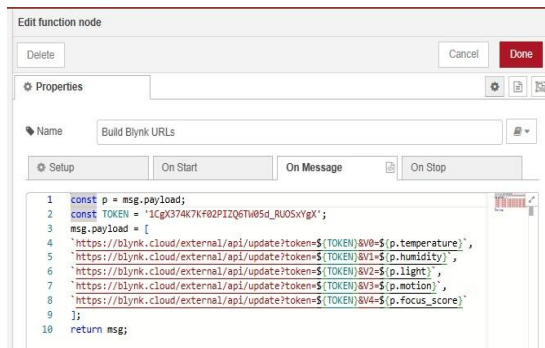
ThingSpeak — ส่งทุก field ใน URL เดียว



api_key + field1..field5 รวมใน URL เดียว

- 1 สร้าง Channel + เพิ่ม 5 fields
- 2 คัดลอก Write API Key
- 3 ใส่ key ใน function (KEY)
- 4 ส่ง HTTP GET request

Blynk — ส่งแยกทีละ Virtual Pin (V0–V4)



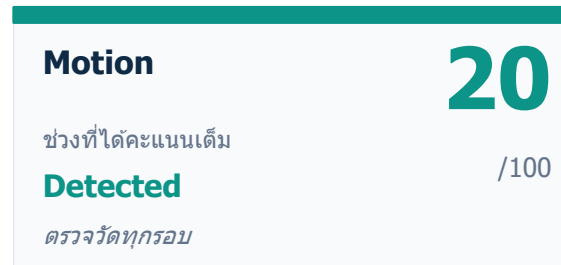
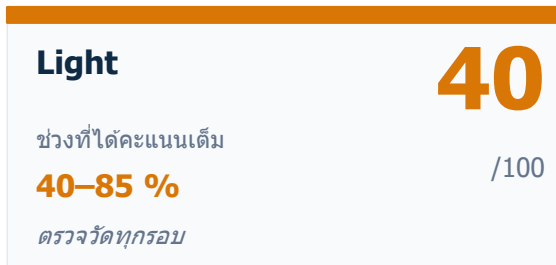
token + V0..V4 ส่งแยก URL — Set URL + GET ทีละตัว

- 1 สร้าง Template + Device
- 2 ตั้ง Datastream V0–V4
- 3 คัดลอก Auth Token
- 4 ใส่ TOKEN + map V0..V4
- 5 Split URLs → ส่งทีละตัว

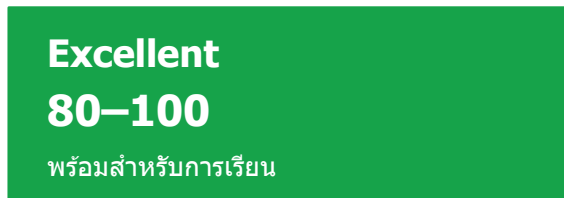
Edge AI / ML Logic

Focus Score + Anomaly Detection แบบง่ายที่ Edge

องค์ประกอบของ Focus Score (เต็ม 100 คะแนน)



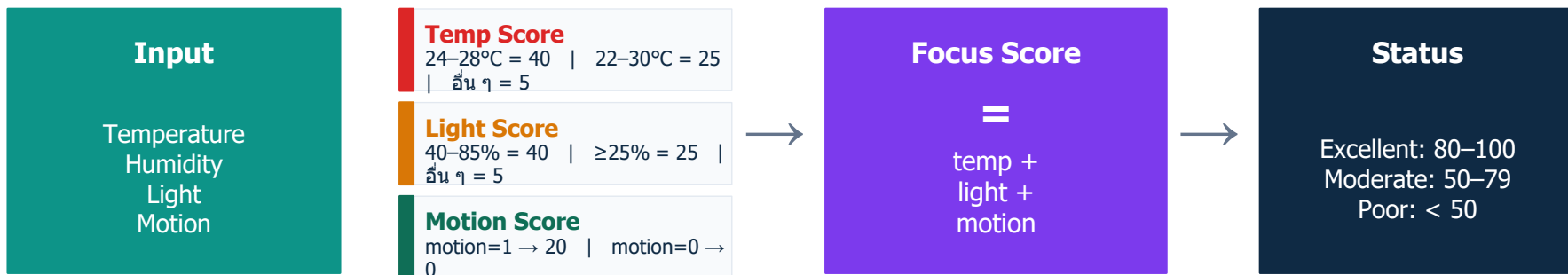
Status Mapping จาก Focus Score



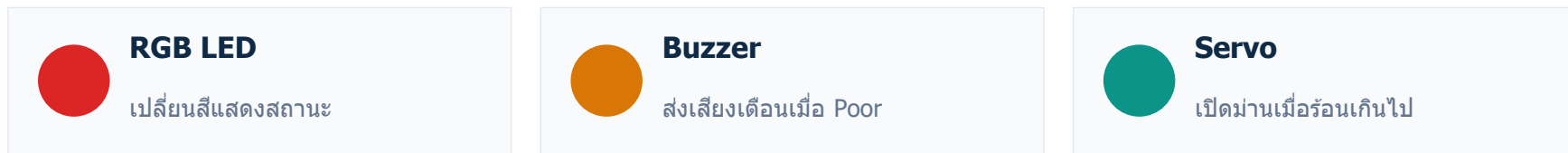
ตัวอย่าง anomaly: $temp > 30^{\circ}C$ หรือ $light < 25\%$ ขณะมีคนอยู่ → แจ้งเตือนทันที

Focus Score Calculation

แปลงค่าจาก sensor เป็นคะแนนสภาพห้องเรียนแบบ real-time



ผลลัพธ์ → ควบคุม Actuator



คำนวณบน Edge (Node-RED) จึงตอบสนองได้รวดเร็ว ไม่ต้องรอ cloud

Anomaly Detection Rules

เงื่อนไขแจ้งเตือนและการตอบสนองของ actuator ที่ Edge

IF **Temperature > 30°C**

THEN **Alert ON + Buzzer ON + Servo OPEN**

เมื่อห้องร้อนเกินจะส่งเสียงเตือนและเปิดม่านเพื่อระบายความร้อน

IF **Light < 25% และ Motion = 1**

THEN **Alert ON + LED RED + Reason: too dark**

เมื่อมีคนในห้องแต่แสงน้อย จะเตือนให้เปิดไฟทันที

IF **Motion = 0 และ Light > 70%**

THEN **Reason only: Light is on but no one detected**

เมื่อห้องว่างแต่ไฟยังเปิดอยู่ จะแจ้งเตือนเพื่อประหยัดพลังงาน

ช่องทางแจ้งเตือน

Local + Remote Notification



Local

RGB LED, Buzzer, Servo ตอบสนองที่ห้องทันที



Remote

Telegram alert ส่งให้ผู้ดูแลเมื่อ alert = true



Cloud

ThingSpeak + Blynk เก็บและแสดงข้อมูล



Edge

ตัดสินใจใน Node-RED ก่อนส่งออก

Edge ML: Trend & Anomaly

Moving Average + Trend Prediction ด้วยข้อมูลย้อนหลัง 5 ค่า

1 Store History

เก็บ temp, humidity, light, focus 5 ค่าหลัง

2 Moving Average

คำนวณ avgTemp, avgFocus จากข้อมูล 5 ค่า

3 Trend Detection

ตรวจ $\text{last.temp} > \text{first.temp} + 1$ หรือ focus ลดลง > 10

4 Anomaly Detection

$|\text{temp} - \text{avgTemp}| > 3 \rightarrow \text{alert} + \text{buzzer} + \text{LED RED}$

5 Prediction + Auto Adjust

Temp rising $\rightarrow$ Servo OPEN / Focus drop $\rightarrow$ LED YELLOW

6 Output Payload

ส่ง avg_temp, avg_focus, trend, prediction, anomaly

ทำไมต้อง Moving Average + Trend Prediction?

ลดสัญญาณรบกวนจากค่าผิดเพี้ยนชั่วคราว และทำนายแนวโน้มล่วงหน้า เช่น ห้องกำลังจะร้อน $\rightarrow$ เปิดมานานก่อนถึงจุด 30°C

Remote Cloud Platforms

ใช้ 2 platforms ตามโจทย์: ThingSpeak + Blynk

ThingSpeak



ใช้สำหรับ

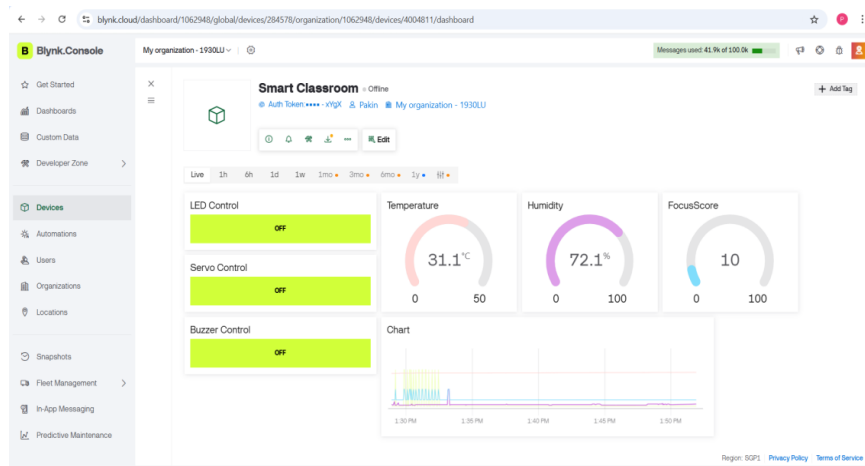
เก็บข้อมูลและแสดงกราฟ time series แบบ public

Channels

5 fields: temp, humidity, light, motion, focus_score

Smart Classroom Focus Monitor

Blynk



ใช้สำหรับ

ควบคุม actuator จากระยะไกล + ดู gauge แบบ live

Virtual Pins

V10 = LED, V11 = Servo, V12 = Buzzer

Local Dashboard Design

Node-RED Dashboard สำหรับใช้งานใน local network

Widgets ในแดชบอร์ด

Temperature Gauge

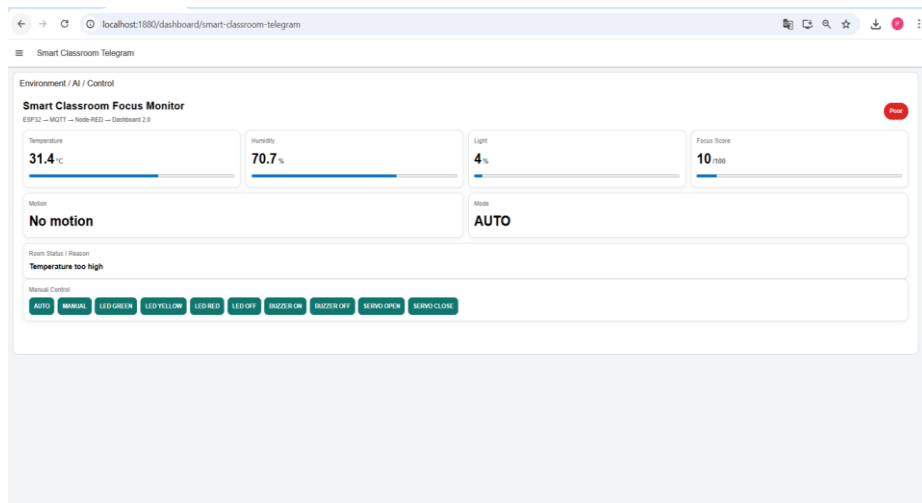
Humidity Gauge

Light Gauge

Focus Score + Status

Temp/Light Charts

Manual Controls



คุณสมบัติเด่น

Auto/Manual switch

เปลี่ยนโหมดได้ทันที

Servo + Buzzer button

ควบคุมแบบ manual

Real-time charts

ดูค่าสดทันที

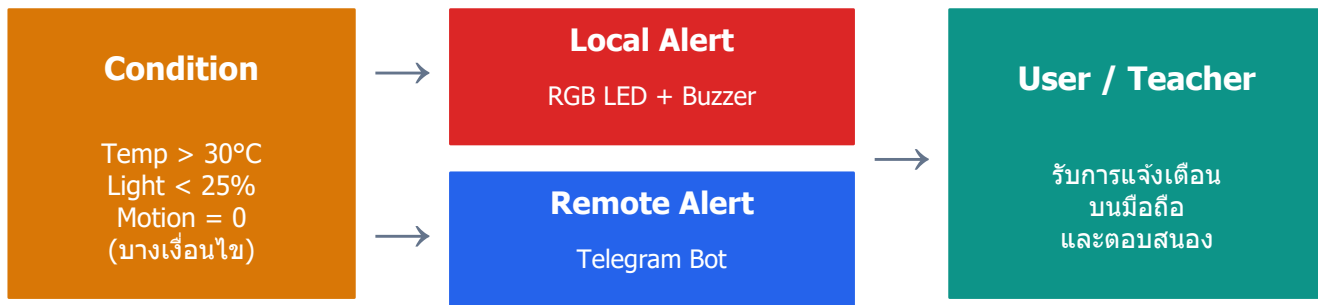
Room Status / Reason

แสดงเหตุผลที่แจ้งเตือน

Notifications

แจ้งเตือนทั้ง *local* และ *remote* ผ่าน *Telegram*

เส้นทางการแจ้งเตือน



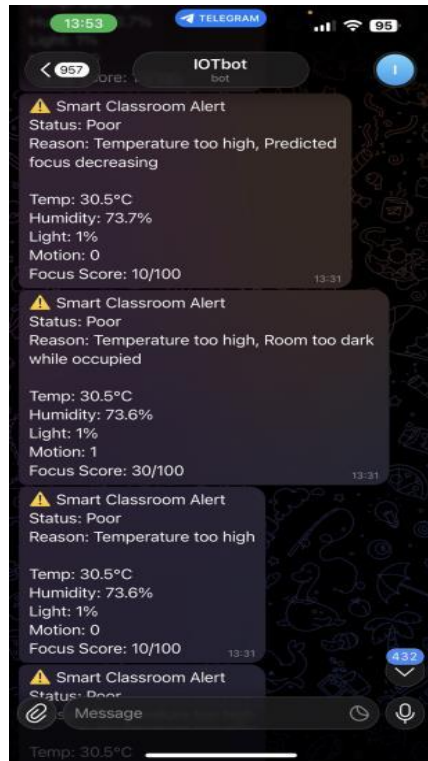
ข้อมูลใน Telegram alert

Status: Poor / Moderate / Excellent

Reason: เช่น Temperature too high, Room too dark

Values: Temp, Humidity, Light, Motion, Focus Score

Prediction: หาก trend แสดงให้เห็นว่าจะแย่ลง



ปัญหาและแนวทางแก้ไข

ข้อจำกัดของโครงการและวิธีที่ใช้แก้ปัญหา

ค่า Sensor แกว่งและไม่นิ่ง

ปัญหา ค่า LDR/DHT22 มี noise และผันผวนเร็วเกินไป

วิธีแก้ ใช้ Moving Average 5 ค่า ลดสัญญาณรบกวน

การเชื่อมต่อ MQTT หลุดบางครั้ง

ปัญหา WiFi ในห้องเรียนอาจไม่เสถียร ทำให้ข้อมูลขาดช่วง

วิธีแก้ ESP32 ทำ auto-reconnect และ retry ทุก 5 วินาที

Edge AI ตัดสินใจผิดเมื่อคนแค่เดินผ่าน

ปัญหา PIR ตรวจจับการเคลื่อนไหวระยะสั้นเป็น motion = 1

วิธีแก้ เพิ่มเงื่อนไข motion ต้อง = 1 ต่อเนื่อง 2 รอบขึ้นไป

Telegram alert ส่งซ้ำๆ มากเกินไป

ปัญหา เงื่อนไขเดิมทำให้แจ้งเตือนทุกรอบที่ค่ายังผิด

วิธีแก้ เพิ่ม cooldown 60 วินาทีหลังจากแจ้งครั้งล่าสุด

ระบบผ่านการทดสอบและพร้อมสำหรับการ Demo

ประโยชน์และการประยุกต์ใช้

Project สามารถนำไปปรับใช้ในงานหลายรูปแบบ

ประโยชน์หลัก



ลดการใช้พลังงาน

ปิดไฟ/แอร์อัตโนมัติเมื่อห้องว่าง



เพิ่มประสิทธิภาพการเรียนรู้

ปรับสภาพห้องให้เหมาะสมตลอดเวลา



ผู้ดูแลมองเห็นภาพรวม

ติดตามหลายห้องพร้อมกันจากที่ใดก็ได้



เก็บข้อมูลเชิงสถิติ

นำไปปรับปรุงตารางเรียนและพลังงานในภายหลัง

การประยุกต์ใช้กับงานอื่น

Smart Office

ตรวจสอบสภาพห้องประชุมและออฟฟิศ

Smart Library

วัดอุณหภูมิ + จำนวนคน เพื่อจัดการแอร์

Smart Lab

เฝ้าระวังสภาพแวดล้อมในห้องวิจัย

Smart Greenhouse

ปรับใช้กับการเกษตรในเรือนปลูกพืช

เพียงเปลี่ยนเซ็นเซอร์และเงื่อนไข ก็นำสถาปัตยกรรมไปใช้กับโดเมนอื่นได้ทันที

การใช้ AI ช่วยในการออกแบบ

ระบบการใช้ AI ในกระบวนการพัฒนาโครงการ

ส่วนที่ AI ช่วยในการออกแบบ

ออกแบบสถาปัตยกรรมระบบ

ใช้ AI ช่วยวาง flow ของ ESP32 ↔ Node-RED ↔ Cloud

เขียนโค้ด ESP32 และ Node-RED

ใช้ AI ช่วย generate code สำหรับ MQTT และ function nodes

ออกแบบ Focus Score และ rules

ปรึกษา AI เพื่อจัดเกณฑ์คะแนนและเงื่อนไข anomaly

หลักการใช้ AI อย่างมีความรับผิดชอบ

- 1 ตรวจสอบโค้ดที่ AI สร้างทุกบรรทัดก่อนใช้งานจริง
- 2 ทดสอบ Focus Score กับข้อมูลจริงในห้องเรียน เพื่อปรับเกณฑ์ให้เหมาะสม
- 3 ปรับ prompt และเงื่อนไข AI ให้สอดคล้องกับโจทย์ที่ได้รับ
- 4 ใช้ AI เป็นผู้ช่วย ไม่ใช่ผู้ตัดสินใจ — นักพัฒนายังต้องเข้าใจระบบเองทั้งหมด

AI เป็นเพียงเครื่องมือ — การออกแบบและการตัดสินใจสุดท้ายคือของผู้พัฒนา